52

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

ABA Problemi & Damgalı Referanslar: AtomicStampedReference vs AtomicMarkableReference

The ABA Problem & Stamped References: AtomicStampedReference vs Markable

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
                    ABA PROBLEMİ VE DAMGA (STAMP) ÇÖZÜMÜ

  1. ABA PROBLEMİ (Bellek Geri Dönüşüm Tuzağı):
     Zaman 0: Thread 1 düğümü okur: Değer = "A"
     Zaman 1: Thread 2 "A"yı "B" yapar.
     Zaman 2: Thread 2 "B"yi tekrar "A" yapar!
     Zaman 3: Thread 1 CAS yapar: "Değer hâlâ A mı?" -> EVET! (CAS BAŞARILI!)
     💥 FACİA: Değer A'dır ama altındaki bağlı liste veya işaretçiler tamamen DEĞİŞMİŞTİR!

  2. DAMGALI ÇÖZÜM (AtomicStampedReference):
     Zaman 0: Değer = "A", Damga = 1
     Zaman 1: Değer = "B", Damga = 2
     Zaman 2: Değer = "A", Damga = 3
     Zaman 3: Thread 1 CAS yapar: "Değer A ve Damga 1 mi?" -> HAYIR (Damga 3!)
     🛡️ CAS BAŞARISIZ OLUR! ABA TUZAĞI ENGELLENDİ!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Masanın üstünde 100 liralık bir banknot bıraktınız. Siz kahve almaya gidince arkadaşınız o parayı alıp kumarda harcadı, sonra cebinden çıkarıp masaya yepyeni başka bir 100 liralık banknot koydu. Döndüğünüzde 'Aaa param burada duruyor' dersiniz ama o para aslında o para değildir! Arada koskoca bir kumar dönmüştür. AtomicStampedReference paranın üzerine görünmez bir versiyon damgası basar; para aynı kalsa bile damga değiştiği için kandırılmazsınız!

🇬🇧 ENGLISH (ELI5)

You leave a $100 bill on your desk. While you grab coffee, a thief steals it, gambles it away, wins, and places a completely different $100 bill back on your desk. You return and see '$100', falsely assuming nothing happened. In concurrent memory, the pointer value returned to 'A', but internal memory references were altered! `AtomicStampedReference` pairs the reference with an incrementing integer version stamp. Even if the value reverts to 'A', the mismatched stamp (version 3 vs version 1) halts the CAS instantly.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ABA Problemi ve Kilit-Siz (Lock-Free) Veri Yapıları: 1) Ortaya Çıkış: Lock-free yığınlarda (Treiber Stack) bir düğüm çıkarılırken (pop) üstteki düğüm A, onun altındaki B'dir. Thread 1, A'yı okur ve B'yi yeni baş yapmayı planlar. Araya giren Thread 2, A'yı ve B'yi çıkarır, sonra hafızadan geri dönüştürülen A nesnesini C'nin üstüne tekrar koyar. Thread 1 uyandığında yığının tepesinde hâlâ A'yı görür ve CAS yapar; yığın kafası B'ye geçer ancak B çoktan silinmiştir! Bellek bozulur. 2) AtomicStampedReference<V>: Referans ile int tipinde bir damgayı (`stamp`) atomik bir çift olarak tutar. `compareAndSet(expectedRef, newRef, expectedStamp, newStamp)` ile hem referansın hem damganın uyuştuğunu doğrular. 3) AtomicMarkableReference<V>: Damga yerine tek bitlik bir boolean (`mark`) tutar; nesnenin silinmek üzere işaretlendiğini (logical deletion) belirtmek için idealdir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ABA Hazard in Lock-Free Data Structures: 1) The Mechanics: In a lock-free Treiber Stack, Thread 1 observes top node `A` pointing to `B`. Before CAS completes, Thread 2 pops `A` and `B`, and then pushes a recycled instance `A` pointing to `C`. Thread 1 resumes and executes `CAS(top, A, B)`. The CAS succeeds because `top == A`, yet node `B` points to unallocated or corrupted memory. 2) `AtomicStampedReference\<V>`: Encapsulates both an object reference and an integer version counter. It invokes a dual-variable CAS (`compareAndSet(expectedRef, newRef, expectedStamp, newStamp)`). 3) `AtomicMarkableReference\<V>`: Pairs a reference with a single boolean mark, optimal for marking nodes as logically deleted before physical linked-list unlinking.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: AtomicStampedReference ile ABA Güvenli Hesap Güncellemesi
import java.util.concurrent.atomic.AtomicStampedReference;

public class AbaPreventionDemo {
    record AccountState(String status) {}

    void main() {
        var initial = new AccountState("ACTIVE");
        int initialStamp = 1;

        // Referans ve başlangıç versiyon damgası
        var accountRef = new AtomicStampedReference<>(initial, initialStamp);

        // Thread 1: O anki referansı ve damgayı okur
        int[] stampHolder = new int[1];
        AccountState observed = accountRef.get(stampHolder);
        int observedStamp = stampHolder[0];

        // Araya giren Thread 2: A -> B -> A döngüsü yapar (Damga artar!)
        var suspended = new AccountState("SUSPENDED");
        accountRef.compareAndSet(observed, suspended, observedStamp, observedStamp + 1);
        accountRef.compareAndSet(suspended, observed, observedStamp + 1, observedStamp + 2);

        System.out.println("Şu anki referans: " + accountRef.getReference().status());
        System.out.println("Şu anki güncel damga: " + accountRef.getStamp());

        // Thread 1 geri döner ve eski damgayla güncellemeye çalışır
        var closed = new AccountState("CLOSED");
        boolean success = accountRef.compareAndSet(observed, closed, observedStamp, observedStamp +
1);

        System.out.println("Thread 1 CAS başarılı oldu mu? " + success + " (ABA Engellendi!)");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

AtomicStampedReference dahili olarak her güncellemede yeni bir küçük nesne (Pair<V, Integer>) oluşturur! Çok yüksek yazma hacimli sistemlerde bu durum GC baskısı yaratabilir. Yüksek performanslı kütüphaneler (ör. Disruptor) damgayı long içinde bit maskeleme ile saklar.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

`AtomicStampedReference` instantiates an internal `Pair\<V, Integer>` node upon state mutations, inducing GC allocation overhead under extreme write contention. Low-latency engines mitigate this by packing references and monotonic counters into a single 64-bit primitive `long` via bit shifting.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• CAS sadece değerin eşitliğini kontrol eder, arada değişip geri gelmesini anlayamaz. Versiyon damgası (stamp) ekleyerek ABA hatasını önleyin.

🎯 Key Takeaways for the Interview

• CAS inspects current value equality, blind to intermediary transitions. Coupling references with monotonic version stamps prevents the ABA trap.